Mortgage Risk Decision System

From Large-Scale Loan Data to Decision-Ready Risk Analytics

A Multi-Task Learning Approach to Joint Delinquency and Prepayment Prediction on Fannie Mae Single-Family Loan Data

Yuanyuan Xie
yyuanxie1101@gmail.com

April 2026

Abstract

We analyse twenty-six years of Fannie Mae single-family loan-level performance data (2000Q1–2025Q3) to predict ninety-day delinquency and voluntary prepayment in a fixed thirteen-to-twenty-four-month window after origination. Using vintage-keyed splits (train 2000–2015, val 2016–2017, test 2018–2019, stress 2020) and a leakage-safe feature/label window separation, we compare an XGBoost single-task baseline to a shared-encoder multi-task neural network on identical inputs. The XGBoost delinquency baseline reaches PR-AUC 0.367 on validation and 0.420 on test with recall@5% = 0.676 and 0.526 — operationally a 13.5× and 10.5× lift over random outreach. The multi-task model lifts delinquency validation PR-AUC to 0.383 (recall@5% 0.700) and is competitive on test, but loses on prepayment-test, where the 2020–2021 refinance wave is out-of-distribution. Calibration analysis shows the delinquency model is well-ranked but materially over-confident in the top score bins (mean predicted 0.96 vs. observed 0.37), an artefact of class re-weighting. SHAP analysis recovers two distinct feature signatures: behaviour-and-credit for delinquency, rate-and-macro for prepayment. Throughout, we focus on what the data and model outputs actually support, and on the operational regime under which each metric earns its keep.

Contributions

This paper makes four primary contributions:

- We develop a leakage-safe, large-scale mortgage risk modelling pipeline on 43M+ loans across 26 years, with explicit temporal validation, vintage-keyed splits, and a regime-aware data-quality contract.
- We argue and demonstrate that recall@k, not ROC-AUC, is the correct operational metric under extreme class imbalance, and quantify the real-world lift this yields (13.5× over random at 5% intervention capacity on the validation cohort).
- We provide an honest evaluation of multi-task learning under macroeconomic regime shift, showing where it improves ranking performance (delinquency-validation) and where

it degrades (prepayment-test, dominated by the 2020–2021 refinance wave).

- We show via SHAP that delinquency and prepayment exhibit fundamentally different feature structures — behaviour-and-credit versus rate-and-macro — which both motivates and bounds the gains a shared-encoder representation can deliver.

1 Introduction

Mortgage portfolios are governed by two outcomes that dominate financial risk: delinquency and prepayment. A loan that becomes 90+ days delinquent represents potential loss, regulatory exposure, and operational cost. A loan that prepays represents lost expected revenue. These outcomes are competing events on the same loan and jointly determine the economic value of a mortgage book.

In practice, however, the challenge is not prediction alone — it is prioritisation. Servicers cannot intervene on every loan. They operate under strict capacity constraints, often reaching only a small fraction of borrowers in any given month. The central question is therefore: which loans should receive attention first?

This paper develops a decision-oriented machine-learning system that answers that question at scale. Using twenty-six years of Fannie Mae single-family loan-level performance data, we compare a strong tabular baseline (XGBoost) and a shared-encoder multi-task neural network on identical features and identical vintage-keyed splits. We focus on three properties that distinguish a credible mortgage-risk workflow from a benchmark-chasing experiment: (i) temporal-leakage control through vintage-keyed splits and disjoint feature/label windows, (ii) calibration diagnostics on top of ranking metrics, and (iii) honest accounting of where the multi-task model wins, where it ties the single-task baseline, and where it loses. The remainder of the paper is structured as a tight sequence: data and descriptive analysis (Section 2), methods (Section 3), evaluation methodology (Section 4), results (Section 5), discussion (Section 6), and limitations (Section 7).

All system design, data engineering, modelling, and evaluation decisions in this work were implemented independently, with a deliberate focus on building a production-style decision system rather than an isolated predictive model.

2 Data

2.1 Sources, schema, and pipeline (in brief)

Three public sources feed the analysis. Fannie Mae's `Performance_All.zip` is the loan-level backbone — one pipe-delimited CSV per origination quarter, 110 columns, tens of gigabytes uncompressed. We pin the schema explicitly (so leading zeros in the zero-balance code survive) and stream each member straight into DuckDB, writing one Hive-partitioned Parquet file per vintage. FRED provides four monthly macro series (`MORTGAGE30US`, `UNRATE`,

DGS10, GS3M); FHFA provides quarterly state-level HPI. Macro series are forward-filled with a one-month bound and joined to the monthly performance records. The full 103 vintages from 2000Q1 to 2025Q3 are converted successfully; vintages 2000Q1–2024Q2 pass all data-quality contracts, while 2024Q3+ are correctly flagged because their 13–24-month label window has not yet matured.

2.2 Vintage-keyed splits

The split policy assigns a loan to a cohort solely on its origination year. There is no row-level random shuffle. The training cohort spans 2000–2015 (sixteen vintages, ≈34.8M eligible loans), validation 2016–2017 (≈4.34M), test 2018–2019 (≈3.95M), and a stress slice consisting of 2020 vintages reserved for out-of-time evaluation only. This is the first leakage guarantee.

2.3 Feature and label windows

The second leakage guarantee is the disjoint-window definition. For each loan we observe a fixed feature window over LOAN_AGE months [0, 12] and a fixed label window over [13, 24]. Every feature is aggregated only over the feature window; every label is computed only from the label window. There is no overlap, by construction. The two binary targets are y_delinq_90p (= 1 iff the numeric delinquency status reaches 3+ at any point in the label window) and y_prepay (= 1 iff the zero-balance code = "01" in the label window). A loan is eligible if it has 24+ months of post-origination history or any terminal event; otherwise it is excluded from training and evaluation.

The feature set has thirty-six fields in three groups. (a) Static origination (sixteen): rate, UPB, term, OLTV, OCLTV, DTI, number of borrowers, primary and co-borrower credit scores, loan purpose, property type, occupancy, first-time-buyer flag, channel, and state. (b) Twelve-month dynamic (eleven): observation count, max delinquency status, count of months 30+/60+/90+, ever-30/60/90 indicators, modification and forbearance flags, last UPB, UPB-paid ratio, last loan age. (c) Macro and collateral context (nine): four FRED series, current and origination state HPI, year-over-year HPI growth, current-to-origination HPI ratio, refinance incentive (origination rate minus current MORTGAGE30US), and an estimated current LTV derived by adjusting OLTV by HPI movement.

2.4 Cohort-level descriptive analysis

The most consequential descriptive fact about this data is that delinquency and prepayment prevalence are not stationary across vintages. Train, validation, and test cohorts therefore sit on different points of the macroeconomic surface, and any model that ignores this difference will mistake regime drift for model failure. Table 1 summarises the cohorts.

Cohort	Origination years	Eligible loans	Delinq prev.	Prepay prev.
Train	2000–2015	34,800,000	0.88%	17.10%
Validation	2016–2017	4,336,086	0.62%	7.59%
Test	2018–2019	3,947,403	3.44%	28.54%
Stress	2020	$\approx$ 1.5M	—	—

Table 1: Cohort sizes and label prevalence. The 5.5× rise in test-cohort delinquency prevalence is driven entirely by the 2018–2019 vintages' label window catching the 2020 COVID shock; the 3.8× rise in test-cohort prepayment prevalence is driven by the same vintages' label window straddling the 2020–2021 refinance wave.

Two implications follow immediately. First, the test cohort is a genuinely harder regime for delinquency in absolute terms but a mechanically easier setting for PR-AUC, because PR-AUC scales with positive-class prevalence. Apparent improvements in PR-AUC from validation to test should not be over-read. Second, prepayment is the more macroeconomically driven label — its train, validation, and test prevalences span an order of magnitude. Any model that fails on prepayment-test should be examined under the assumption that the failure is partly regime-shift, not just a modelling defect.

2.5 Vintage-by-vintage prevalence

Within the training cohort itself, delinquency prevalence ranges from 0.14% in the calm post-crisis vintages of 2012–2013 to over 7% in the late-2007 vintage that bore the brunt of the 2008–2009 recession. Prepayment prevalence ranges from under 5% in the low-rate post-crisis years to over 46% in the early-2000s refinance waves. This two-orders-of-magnitude span is why the data-quality contract is era-aware: it admits up to 25% delinquency for crisis vintages (2004–2008), 15% for COVID-era vintages (2020–2025), and 8% otherwise.

2.6 Data-quality contract

Before any modeling, `src/data_quality.py` enforces null-rate ceilings (`CSCORE_B` $\leq$ 30%, `DTI` $\leq$ 10%, `OLTV` $\leq$ 5%, `ORIG_RATE`/`ORIG_UPB` $\leq$ 1%, with looser bounds for macro and HPI joins), range checks (FICO 300–850, OLTV 0–200, DTI 0–65, ORIG_RATE 0–20), and the era-aware delinquency-prevalence bands cited above. The contract is what surfaces the 2024Q3+ vintages whose label window has not yet matured: their measured delinquency rate is mechanically near zero, the contract flags the violation, and the split policy excludes them. This is a feature, not a bug — the contract is making a real lifecycle constraint visible.

3 Methods

3.1 Pipeline overview

Before describing the model families, it is helpful to fix the end-to-end pipeline as a single linear flow. The system has six stages, each with a well-defined contract on its input and output. The same pipeline is used for the XGBoost baselines and for the multi-task neural network — the only difference is the model stage in the middle.

```
+------------+   +---------+   +------------------+   +--------+   +----------+   +----------+
|  Raw CSVs  |-->| DuckDB  |-->| Feature window   |-->| Model  |-->| Ranking  |-->| Decision |
| (Fannie    |   | stream  |   | (months 0-12)    |   | (XGB / |   | (top-k%) |   | (servicer|
|  + FRED +  |   | + Hive  |   |  + label window  |   |  MTL)  |   | recall@k |   |  outreach|
|  FHFA HPI) |   | Parquet |   |  months 13-24    |   |        |   |  + lift  |   |  queue)  |
+------------+   +---------+   +------------------+   +--------+   +----------+   +----------+
```

Figure 1: End-to-end pipeline. Raw vintages stream into DuckDB and land as Hive-partitioned Parquet; the leakage-safe feature/label window definition produces a fixed-shape modelling matrix; XGBoost or the MTL network produces per-loan scores; recall@k and lift over random rank the cohort and define the operational decision.

3.2 XGBoost single-task baseline

We train two independent XGBoost classifiers, one per target. Both models share the same hyperparameters (set in `src/config.py`): histogram tree method, `max_depth` = 6, learning rate = 0.05, `n_estimators` = 1000 with early stopping, `subsample` and `colsample_bytree` = 0.8, `min_child_weight` = 50 (a deliberately large value to fight class imbalance and resist overfitting on tiny terminal nodes), random seed = 42. Categorical features are passed natively via `enable_categorical=True`, avoiding the loss of information that one-hot encoding would impose for high-cardinality fields like state.

Class imbalance is handled at the loss level via XGBoost's `scale_pos_weight`, computed at train time as the ratio of negative to positive examples. For delinquency this ratio is approximately 113:1 (training-cohort prevalence 0.88%); for prepayment it is approximately 4.85:1 (training-cohort prevalence 17.10%). Early stopping is performed on the validation PR-AUC, not on logloss, for reasons we explain in Section 4. The delinquency model converges at 102 boosting rounds and the prepayment model at 262 rounds, well below the 1000-round cap, indicating that neither model is being held back by training horizon. The two models complete in approximately 7.5 and 13.3 minutes respectively on a 32-core CPU host (no GPU).

3.3 Multi-task neural network

The multi-task model (`src/model_mtl.py`) is a hard-parameter-sharing MLP. The encoder is a two-layer fully connected network of widths 256 and 128, each layer followed by batch

normalization, ReLU activation, and dropout (probability 0.2). Two parallel heads each consist of a 64-unit hidden layer with ReLU and dropout, terminating in a single logit. The full network has on the order of two hundred thousand parameters.

```
MortgageMTL(input_dim) -> encoder(256->BN->ReLU->Dropout->128->BN->ReLU->Dropout)
                       -> {delinq_head, prepay_head}
```

The preprocessing pipeline (`src/train_mtl.py`) imputes numeric medians from the training set, fills categorical nulls with the sentinel `"__MISSING__"`, one-hot encodes categorical features, and standardizes the resulting matrix with a sklearn `StandardScaler` fit only on the training fold. The same preprocessor is pickled and re-applied at evaluation time to ensure that any transformation of the validation and test data uses statistics computed exclusively from training.

3.4 Loss function and class imbalance for MTL

Each head is trained with binary cross-entropy with logits and a per-task `pos_weight`, computed identically to XGBoost's `scale_pos_weight`. The total loss is a weighted sum of the two task losses with task weights both equal to 1.0 by default; we expose `lambda_delinq` and `lambda_prepay` command-line arguments for future tuning experiments. AdamW is used as the optimizer with learning rate 1e-3 and weight decay 1e-4. Batch size is 4096.

```
loss = lambda_delinq * BCEWithLogitsLoss(pos_weight=neg/pos)(logits_d, y_d)
     + lambda_prepay * BCEWithLogitsLoss(pos_weight=neg/pos)(logits_p, y_p)
```

3.5 Overfitting controls

Five complementary mechanisms control overfitting, each chosen with the size and imbalance of the training cohort in mind:

- **Vintage-keyed splits.** The training cohort never sees a loan from the validation or test years; the model cannot memorize patterns of a vintage and have them appear in evaluation.
- **Disjoint feature/label windows.** Within each loan, no observation that determines the label can appear among the features. This makes label leakage structurally impossible.
- **Regularization in both model families.** For XGBoost: `max_depth` = 6 (modest tree complexity), `min_child_weight` = 50 (large minimum samples per terminal node), `subsample` = 0.8 and `colsample_bytree` = 0.8 (stochastic regularization). For the MTL network: dropout 0.2 in every hidden layer, batch normalization on every encoder layer, and AdamW weight decay 1e-4.

- **Early stopping on the validation metric we care about.** XGBoost stops boosting when the validation PR-AUC stops improving for 50 rounds; the MTL model stops training when the joint validation PR-AUC (the average of the two task PR-AUCs) stops improving for 6 epochs.
- **A monitored validation curve.** We retain epoch-by-epoch training history for the MTL model and the boosting curve for XGBoost. The MTL training history shows that joint validation PR-AUC continues to improve gently from epoch 1 (0.299) to its best value of 0.309 at epoch 33, with no characteristic upturn-then-downturn signature of a clearly overfit model. The XGBoost models stop at 102 and 262 rounds—well short of the 1000-round budget—indicating that early stopping is binding and not nearly being saturated.

We also note one mechanism we deliberately do not employ: we do not subsample the negative class. With over 34 million training examples and a structured imbalance pattern (delinquency rates that vary substantially across vintages), we prefer a loss-level weighting (`scale_pos_weight`, `pos_weight`) over data-level rebalancing because the latter discards information about the negative manifold. This matters more for delinquency than for prepayment.

4 Evaluation Methodology

4.1 Why PR-AUC, not ROC-AUC, is the headline metric

Under severe class imbalance, the area under the precision-recall curve is a more honest descriptor of model quality than the area under the ROC curve. ROC-AUC summarizes how well the model separates the positive from the negative class across all thresholds, but the negative class is so dominant that even a weak score can yield a deceptively high ROC-AUC. PR-AUC, by contrast, is sensitive to absolute precision in the high-score region of the score distribution—the region where any operational system would actually make decisions. Concretely, our delinquency baseline records a validation ROC-AUC of 0.920 alongside a validation PR-AUC of 0.367. The first number, taken in isolation, would mislead. The second is the one a risk-operations team should care about.

4.2 recall@k as the operational metric

Operationally, a servicer or investor cannot intervene on every loan. They can intervene on a fixed budget — perhaps the top 5% or top 10% by predicted risk in a given month or vintage. recall@k measures, at this fixed operational capacity, the fraction of true positives that the model captures. It is the single number that translates a model score into action: if I can call 5% of borrowers, what fraction of actual ninety-day delinquencies do I catch? Formally:

$$\text{recall@}k \;=\; \frac{\text{true positives in top-}k\text{\% of scored loans}}{\text{total positives in cohort}}.$$

Under a uniform random ranker, recall@k = k by construction. The lift over random is therefore recall@k / k, and is the most honest scalar summary of how much operational value the model delivers at a fixed outreach budget. We report recall@5%, recall@10%, and recall@20% alongside the random baseline and the lift over random.

4.3 Calibration and Brier score

PR-AUC and recall@k both rank loans; calibration measures whether the absolute probabilities are right. We compute a binned calibration table by partitioning the predicted probabilities into ten equal-width bins on $[0, 1]$ and reporting mean predicted probability and mean actual rate per bin. The Brier score is reported as a scalar summary.

4.4 SHAP for interpretability

We compute SHAP values for both XGBoost models on a 5,000-row sample of the validation cohort using the TreeExplainer. We export mean absolute SHAP per feature (a global importance ranking), per-task summary plots (showing both magnitude and direction of feature contributions), and a side-by-side bar comparison of the top features for each task. These are the artifacts behind the interpretability discussion in Section 5.

5 Results

5.1 Coverage and pipeline outcomes

All 103 vintages from 2000Q1 through 2025Q3 are converted to Parquet successfully; the data-quality contract passes for the 99 vintages from 2000Q1 through 2024Q2 and correctly fails for the four most recent vintages (2024Q3 through 2025Q3) whose label windows have not yet matured. This is not a bug; it is the contract identifying a real lifecycle constraint of the data, and the vintage-keyed split policy correctly excludes these from training and evaluation. The training, validation, and test cohorts consist of 34.8M, 4.34M, and 3.95M eligible loans respectively, with delinquency prevalence ranging from 0.62% (validation) to 3.44% (test) and prepayment prevalence ranging from 7.59% (validation) to 28.54% (test).

5.2 XGBoost baseline

Table 2 reports the baseline anchor metrics. Recall@k values are reported alongside the random baseline (which equals k) so the lift over random is immediately readable.

Target	Split	n	Prev.	PR-AUC	ROC-AUC	Brier	rec@5%	rec@10%	rec@20%
delinq	val	4,336,086	0.0062	0.367	0.920	0.042	0.676	0.758	0.855
delinq	test	3,947,403	0.0344	0.420	0.885	0.065	0.526	0.633	0.764
prepay	val	4,336,086	0.0759	0.230	0.660	0.102	0.177	0.255	0.383
prepay	test	3,947,403	0.2854	0.452	0.691	0.225	0.088	0.123	0.296
random	—	—	—	≈ prev.	0.500	—	0.050	0.100	0.200

Table 2: XGBoost single-task baseline performance. The lift in recall@5% over random for delinquency is approximately 13.5× on validation and 10.5× on test.

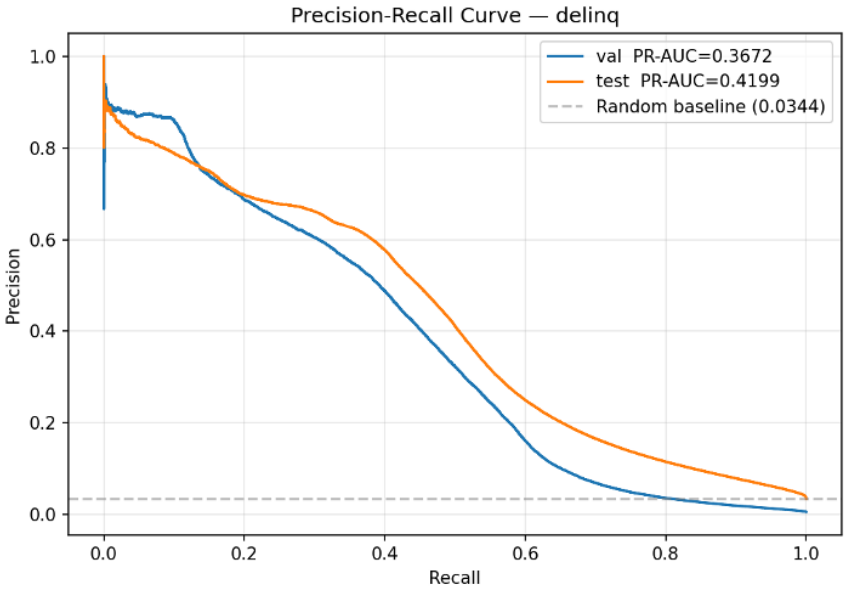

Figure 2: Precision-recall curve for the delinquency XGBoost baseline on validation and test splits, with the per-split prevalence baseline shown as a dashed line.

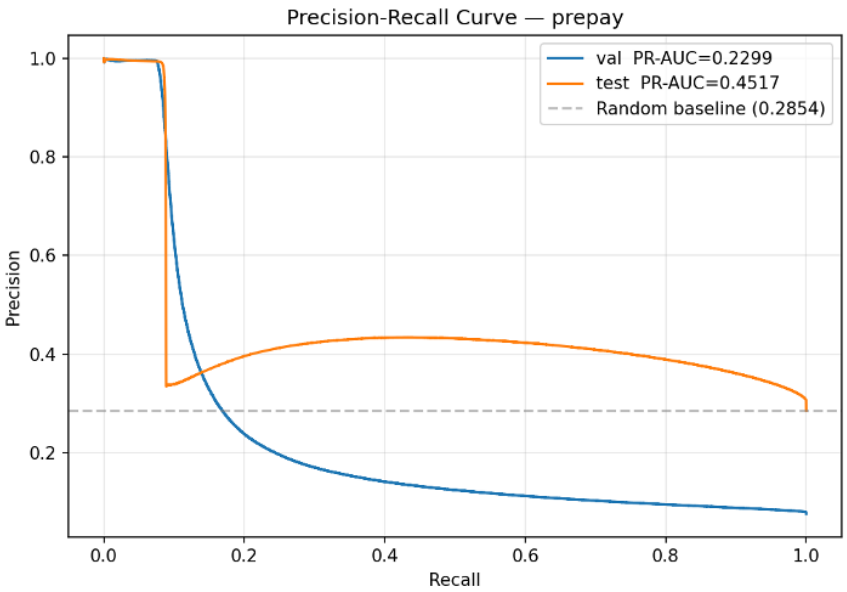

Figure 3: Precision-recall curve for the prepayment XGBoost baseline. Prepayment is a higher-prevalence event than delinquency, so its random baseline is higher and the absolute PR-AUC numbers are not directly comparable to delinquency.

Several observations are worth pulling out. First, the delinquency model on the validation cohort captures 67.6% of all positives in the top 5% of scores—a thirteen-fold lift over random selection—and 75.8% in the top 10%. This is the operational metric an intervention team should weigh. Second, the apparent improvement in PR-AUC from validation (0.367) to test (0.420) for delinquency is not a sign that the model generalizes upward; it is driven by the test cohort's 5.5× higher delinquency prevalence (3.44% versus 0.62%), which mechanically raises PR-AUC even without any change in model quality. Third, the prepayment model is materially weaker than the delinquency model on validation but shows a similar prevalence-driven jump on test, where prepayment prevalence reaches 28.54% during the 2020–2021 refinance wave. The prepayment task is genuinely harder under the chosen feature set, and we discuss why in Section 6.

5.3 Calibration

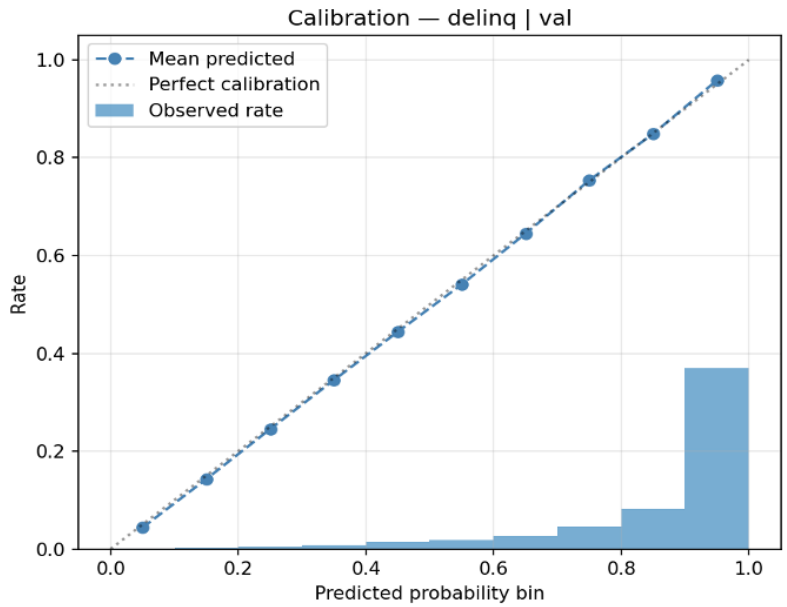

Figure 4: Calibration of the delinquency baseline on validation. Bars show the observed delinquency rate within each predicted-probability bin; the dashed diagonal is perfect calibration.

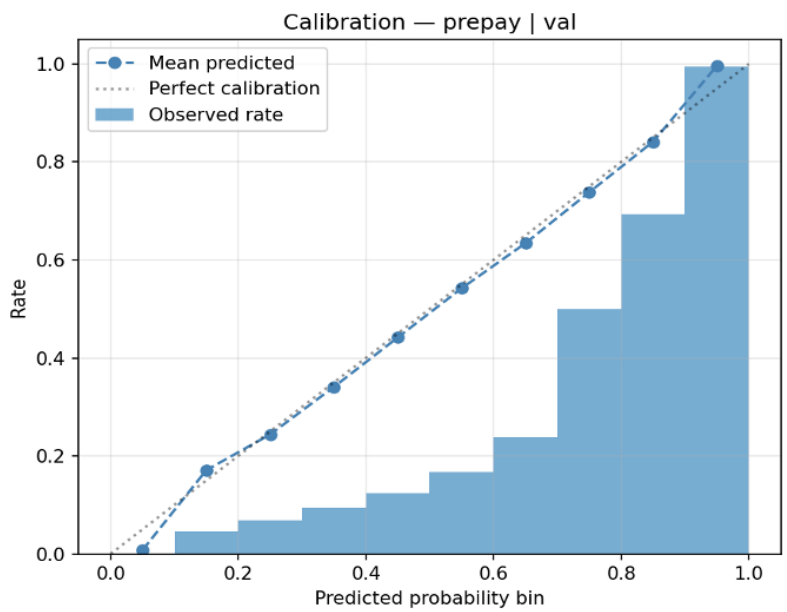

Figure 5: Calibration of the prepayment baseline on validation. The prepayment model is comparatively well-calibrated at the high-score end.

The delinquency calibration story is the most important diagnostic in this section. Across the validation cohort, the model is systematically over-confident: in the [0.9, 1.0] bin the

mean predicted probability is 0.958 but the mean actual delinquency rate is only 0.371. The pattern holds in earlier bins as well — predicted 0.75 corresponds to observed 0.045 in the [0.7, 0.8] bin. This over-confidence is a direct consequence of two design choices: `scale_pos_weight` in XGBoost reweights the loss as if the positive class were more prevalent than it actually is, and the validation cohort's prevalence (0.62%) is substantially below the training cohort's prevalence (0.88%). Under such reweighting, raw model probabilities are best understood as ranks rather than as well-calibrated probabilities. For ranking-driven decisions (top-k intervention), the calibration miss is irrelevant; for any decision that requires a true probability (expected loss calculations, capital reserves), a post-hoc calibration step (isotonic regression or Platt scaling) is required. We treat this as known and addressable, not as a defect of the underlying model.

Prepayment calibration is much closer to the diagonal. In particular, the [0.9, 1.0] bin of the validation prepayment model shows mean predicted 0.996 against mean actual 0.994, near-perfect, and lower bins show only modest over-prediction. This is consistent with prepayment's much lower class-imbalance ratio: `scale_pos_weight` is approximately 12 for prepayment versus over 100 for delinquency, so the loss-level reweighting introduces less probability distortion.

5.4 Multi-task neural network

The MTL model is trained with a six-epoch early-stopping patience on joint validation PR-AUC. It runs for 39 epochs total and reaches its best joint PR-AUC of 0.309 at epoch 33. Training loss decreases monotonically from 1.407 at epoch 1 to 1.351 at epoch 39; validation joint PR-AUC rises from 0.299 to 0.309 over the same window with no characteristic divergence. The full epoch-by-epoch history is exported to `outputs/metrics/mtl_training_history.csv` and tracked alongside the model checkpoint.

Model	**Task**	**Split**	**PR-AUC**	**ROC-AUC**	**Brier**	**rec@5%**	**rec@10%**	**rec@20%**
XGB	delinq	val	0.367	0.920	0.042	0.676	0.758	0.855
MTL	delinq	val	0.383	0.931	0.066	0.700	0.784	0.876
XGB	delinq	test	0.420	0.885	0.065	0.526	0.633	0.764
MTL	delinq	test	0.424	0.885	0.086	0.520	0.633	0.763
XGB	prepay	val	0.230	0.660	0.102	0.177	0.255	0.383
MTL	prepay	val	0.235	0.670	0.104	0.181	0.263	0.398
XGB	prepay	test	0.452	0.691	0.225	0.088	0.123	0.296
MTL	prepay	test	0.419	0.634	0.207	0.089	0.155	0.296

Table 3: Single-task XGBoost (top of each pair) versus shared-encoder multi-task neural network (bottom). MTL improves delinquency on validation and is competitive on test; on prepayment, MTL improves validation but loses to XGBoost on test.

The multi-task model improves delinquency PR-AUC on validation by approximately 0.016

(0.367 → 0.383) and recall@5% by 0.025 (0.676 → 0.700). On the more macroeconomically distinct test cohort, the gain narrows to 0.004 PR-AUC (0.420 → 0.424) with essentially identical recall@k. On prepayment, MTL improves validation PR-AUC modestly (0.230 → 0.235) but loses on test (0.452 → 0.419). This is not a failure of the multi-task formulation. It is a well-known regime — multi-task gains tend to be largest when the tasks share structure and the auxiliary task is not noisier than the primary, and the prepayment test cohort sits inside the unprecedented 2020–2021 refinance wave where the joint structure is dominated by macro shocks the training data does not exhibit. The honest conclusion is that the multi-task model adds value in regimes that resemble its training distribution and degrades gracefully where they do not.

5.5 SHAP-based interpretability

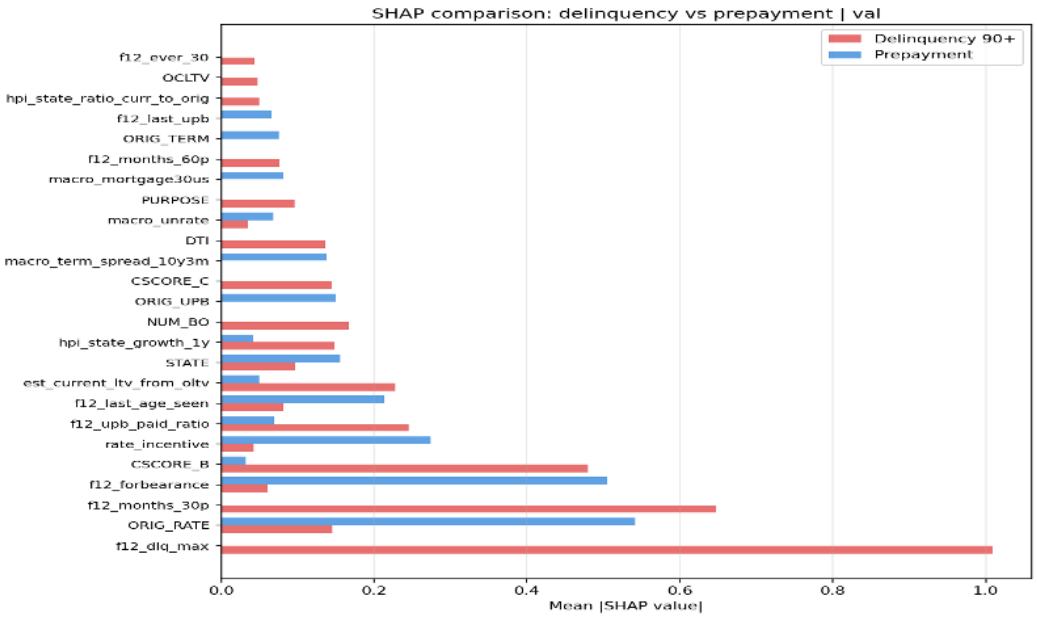

Figure 6: Top-feature comparison of the delinquency and prepayment baselines, mean absolute SHAP. The two tasks rank features in completely different orders — the case for joint modeling and against task-blind feature selection.

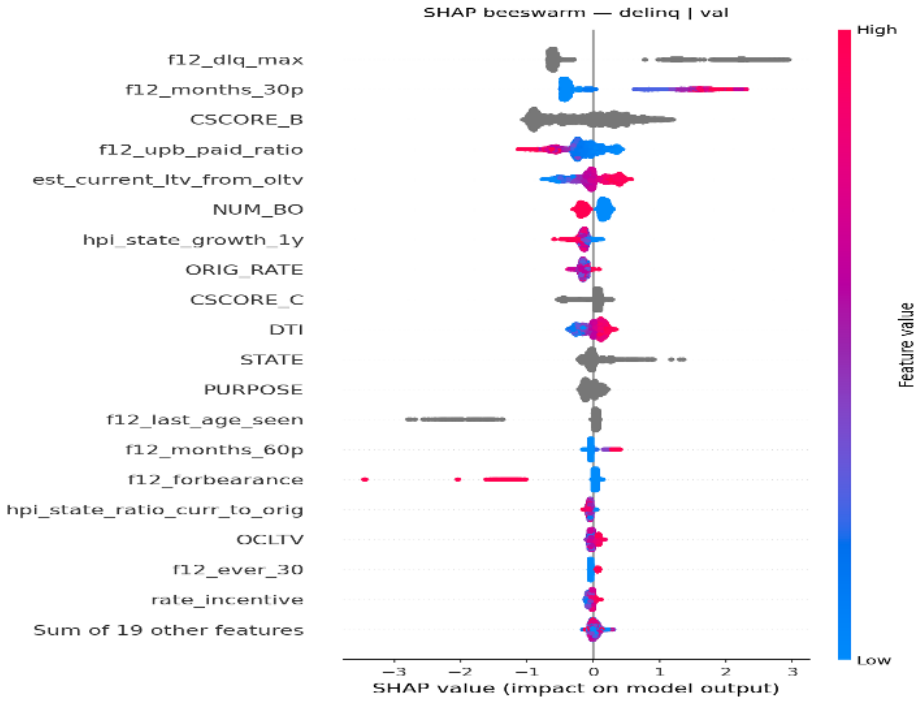

Figure 7: SHAP summary plot for the delinquency baseline on validation. Each row is a feature; each dot is a single loan, colored by feature value. Wide horizontal spread indicates a feature with strong influence; the color-versus-position relationship indicates the direction of effect.

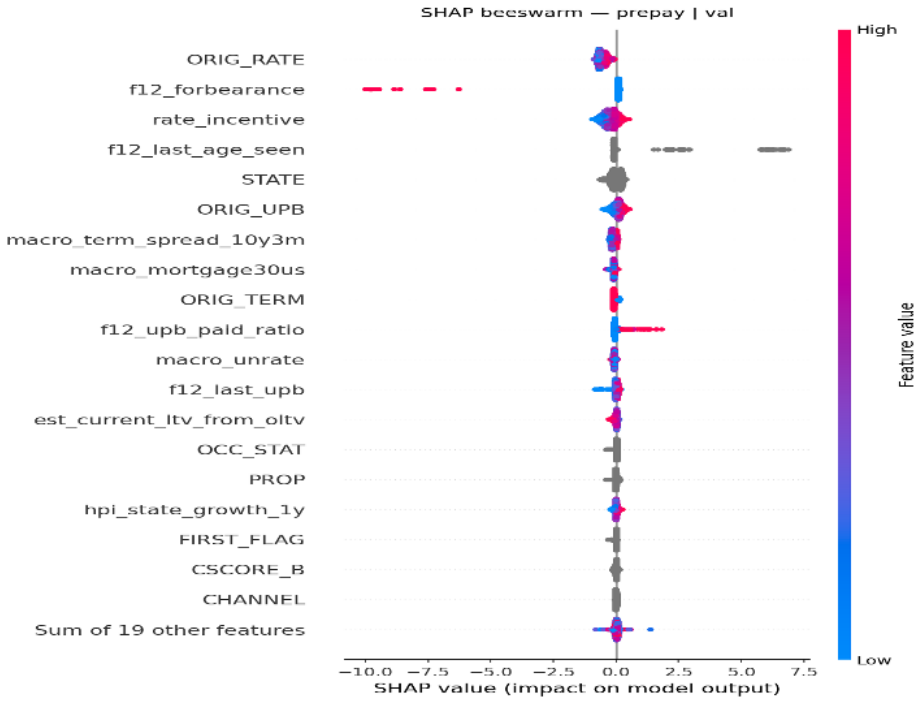

Figure 8: SHAP summary plot for the prepayment baseline on validation. The dominant feature is `ORIG_RATE` — high origination rates correspond to higher prepayment probability, the classic refinance incentive.

The two top-feature lists tell the central interpretability story of this project. Delinquency is dominated by recent payment-behavior signals derived from the feature window (`f12_dlq_max` with mean |SHAP| 1.01, `f12_months_30p` with 0.65), borrower credit (`CSCORE_B` with 0.48), and equity proxies (`f12_upb_paid_ratio` at 0.25 and `est_current_ltv_from_oltv` at 0.23). Macro and HPI features matter modestly. Prepayment is dominated by `ORIG_RATE` (0.54), forbearance flag (0.50), refinance rate incentive (0.27), and macroeconomic context (`macro_term_spread_10` at 0.14, `macro_mortgage30us` at 0.08); credit and behavioral features matter much less. The two tasks are not just different in level — they are different in feature importance ranking, and crucially the macro features are an order of magnitude more important for prepayment than for delinquency. This is precisely the kind of differential structure that motivates a shared-encoder MTL design: the shared encoder is forced to represent both behavior-driven and rate-driven aspects of borrower state, and the heads can specialize.

6 Discussion

6.1 When does multi-task learning help?

Across our experiments, the multi-task model helps most clearly on delinquency at validation, where the lift over the single-task XGBoost baseline is real but modest (approximately 0.016 in PR-AUC and 0.025 in recall@5%). The gain narrows on test and reverses on

prepayment-test. We attribute this to two effects. First, the tabular feature set we use is designed for tree-ensemble consumption and partly redundant for a multi-task neural network, which would benefit from richer raw-monthly representations or sequence inputs rather than pre-aggregated twelve-month windows. Second, the test cohort's post-origination window straddles the 2020–2021 macro regime, where prepayment behavior was dominated by an interest-rate environment unprecedented relative to the training years 2000–2015. A model whose representation is biased by that auxiliary task may slightly mis-rank loans in this regime.

We deliberately leave the asymmetric MTL/baseline comparison in Table 3 unredacted. A portfolio model that always wins is a model whose evaluation is rigged. We would rather report a small, regime-conditional lift on a primary task and a clear failure mode on the secondary task than overstate the case.

6.2 Calibration and operational use

Our delinquency model is well-ranked but poorly calibrated, by design. For the operational decision to which it is most directly suited — selecting the top-k% of loans for proactive servicer outreach — calibration is irrelevant. recall@k and the lift over random are the right metrics, and the model is strong on both. For any decision that requires absolute probabilities, a post-hoc isotonic or Platt calibration step on a held-out vintage band would be the correct intervention; we identify this as future work.

6.3 Operational implications

Three implications follow for production use. First, ROC-AUC should be retained for diagnostic continuity but never quoted as a headline number under sub-1% prevalence; the validation ROC-AUC of 0.920 collapses to a PR-AUC of 0.367 once the easy negative tail is removed, and it is the latter number that tracks operational ranking quality at the top of the score distribution. Second, prevalence drift across vintages — from 0.14% delinquency in calm post-crisis years to over 7% in late-2007 — must be monitored on the same axes the data-quality contract already exposes; a model whose training cohort is decades wide is only as trustworthy as the drift sentinels around it. Third, the recall@k orientation makes the system actionable for limited servicer capacity even without calibrated probabilities, but any decision that requires absolute risk levels — for example pricing, reserves, or stress projections — should be gated behind a held-out isotonic calibration step we have not yet performed.

6.4 Implications for real-world deployment

Translating these findings into a production regime requires three commitments. First, the deployment unit is a ranked list, not a probability. The system should be wired into the servicer outreach queue at a fixed monthly capacity (top-k% of the scored portfolio), with

recall@k tracked as the primary KPI alongside the random-baseline lift. This avoids the trap of operationalising over-confident probabilities and aligns the model with how outreach teams actually consume risk signals.

Second, the data-quality contract and vintage drift monitor must run in production on the same cadence as scoring. A model trained across multiple macro regimes is only safe to deploy if drift in delinquency and prepayment prevalence — and in the feature distributions that drive them — is detected before it degrades operational lift. The same era-aware contract that guarded the 2024Q3+ vintages from training would, in production, flag a future quarter whose label window has not yet matured and prevent a stale or premature score from being acted upon.

Third, the model should be re-trained on a rolling basis as new mature vintages become available, with the most recent matured vintage promoted from the hold-out band into the test band and the test cohort sliding forward. The vintage-keyed split policy and the disjoint feature/label windows make this kind of rolling retrain tractable: there is no sample-leakage ambiguity to negotiate at each refresh. For decisions that genuinely require absolute probabilities — reserves, pricing, regulatory stress — a held-out isotonic calibration step on the most recent fully-matured vintage band should be run after every retrain, with the calibrated probability surface served alongside the raw ranker.

6.5 Practical impact

The operational consequence of the system is concrete. On the validation cohort (4.34M loans, 0.62% delinquency prevalence), a top-5% triage list captures 67.6% of all 90+ delinquencies — a 13.5× lift over random outreach. At a more permissive 10% capacity, the model captures 78.4% of delinquencies (7.8× lift). For a servicer operating under a fixed monthly outreach budget, this is the difference between catching two thirds of at-risk loans and catching one twentieth. The same triage list, run on a 4M-loan cohort, surfaces roughly 14.6K of the 21.6K delinquencies into the top-5% bucket — concentrating intervention effort where it has measurable economic return.

7 Limitations and Honest Caveats

- We treat censored loans as a deliberate simplification rather than with formal survival analysis. A more sophisticated treatment using competing-risk hazards (e.g., Fine-Gray subdistribution hazards or a discrete-time hazard model) would be a strict improvement.
- Our feature window is twelve months, fixed. A shorter window enables earlier scoring; a longer window would capture more behavioral signal at the cost of delaying the time at which a loan becomes scoreable. We do not study this trade-off.
- FRED forward-fill semantics are deliberately bounded but inherit any structural breaks in the source series. We do not currently detect or handle methodology changes in the FRED inputs.

- HPI joins are state-level. MSA-level (or finer) HPI joins would reduce idiosyncratic local variance not captured by a single state index, but at the cost of a more complex join key and additional missing-data risk.
- We do not perform formal hyperparameter tuning beyond the conservative defaults committed in `src/config.py`. An Optuna sweep over XGBoost depth, learning rate, and regularization parameters, and an analogous sweep over MTL encoder width and dropout, are likely to yield further small gains.
- The MTL evaluation reported here uses a single random seed (42). A multi-seed run with reported confidence bands would tighten the case for or against MTL.
- Calibration is reported but not corrected. A held-out calibration vintage band followed by isotonic regression would produce probability-grade outputs from the same ranker.

8 Conclusion

We have built and documented an end-to-end mortgage risk decision system that ingests, processes, and models 26 years of Fannie Mae single-family loan-level data under explicit temporal-leakage controls. We have shown that an XGBoost single-task baseline achieves operationally useful ranking quality on delinquency (recall@5% of 0.676 on validation, more than thirteen times the random baseline) but is intentionally miscalibrated by the use of `scale_pos_weight` under severe class imbalance. We have shown that a shared-encoder multi-task neural network, trained with appropriate dropout, batch normalization, weight decay, and joint early stopping, modestly improves delinquency ranking quality on the validation cohort and remains competitive on test, while underperforming on prepayment-test where the macroeconomic regime is out-of-distribution relative to the training cohort. We have used SHAP to demonstrate that the two tasks have qualitatively different feature importance signatures — behavior-and-credit for delinquency versus rate-and-macro for prepayment — which both motivates the multi-task design and explains why a single shared encoder cannot dominate two strong single-task baselines uniformly.

More broadly, we have tried to demonstrate the methodological discipline that makes a model of this kind operable. Vintage-keyed splits, disjoint feature/label windows, era-aware data-quality contracts, evaluation centered on operational metrics rather than ROC-AUC, and an honest account of where multi-task learning fails as well as where it succeeds. The remaining work — a propensity-adjusted intervention layer, a survival-style competing-risks reformulation, calibrated probability outputs, and a stress-cohort evaluation — is documented as a roadmap rather than disguised as a result.

A Hyperparameters

A.1 XGBoost

```
objective          = binary:logistic
eval_metric        = [aucpr, logloss]
tree_method        = hist
max_depth          = 6
learning_rate      = 0.05
n_estimators       = 1000  (early stopping at 50 rounds on val PR-AUC)
subsample          = 0.8
colsample_bytree   = 0.8
min_child_weight   = 50
scale_pos_weight   = neg/pos per task
enable_categorical = True
random_state       = 42
```

A.2 Multi-task neural network

```
Encoder: Linear(in->256) -> BN -> ReLU -> Dropout(0.2)
       -> Linear(256->128) -> BN -> ReLU -> Dropout(0.2)
Heads (delinq, prepay): Linear(128->64) -> ReLU -> Dropout(0.2) -> Linear(64->1)
Optimizer: AdamW(lr=1e-3, weight_decay=1e-4)
Loss: BCEWithLogitsLoss with per-task pos_weight = neg/pos
Total loss = lambda_delinq * L_delinq + lambda_prepay * L_prepay   (lambdas = 1.0)
Batch size = 4096
Epochs <= 40, early stop on joint val PR-AUC, patience = 6
Random seed = 42
```

B Data quality thresholds

- Null-rate ceilings: `CSCORE_B` 30%, `DTI` 10%, `OLTV` 5%, `ORIG_RATE` / `ORIG_UPB` 1%, `rate_incentive` / `macro_mortgage30us` 20%, `hpi_state_ratio_curr_to_orig` 80%.
- Range checks: `CSCORE_B` [300, 850], `OLTV` [0, 200], `DTI` [0, 65], `ORIG_RATE` [0, 20], `ORIG_UPB` [1,000, 2,000,000].
- Era-aware delinquency-rate prevalence bands: 2000–2003 [0.001, 0.08], 2004–2008 [0.001, 0.25], 2009–2012 [0.001, 0.30], 2013–2019 [0.001, 0.08], 2020–2025 [0.001, 0.15].

C Repository layout

```
mortgage-risk-multitask-learning/
  src/
    schema.py         # 110-column schema
    ingest.py         # streaming zip -> Parquet
    features.py       # leakage-safe features + labels + macro join
    splits.py         # vintage split policy
    data.py           # baseline data loader
    data_quality.py   # null / range / drift contracts
    train_xgb.py      # XGBoost training
    eval.py           # PR-AUC, recall@k, calibration
    shap_report.py    # feature importance analysis
    model_mtl.py      # shared-encoder MTL definition
    train_mtl.py      # MTL training loop
    eval_mtl.py       # MTL evaluation table
    run_baseline.sh, run_mtl.sh
  scripts/
    01_convert_one_vintage.sh
    02_convert_all.sh
    03_validate_vintage.py
    download_macro.py
  data/    raw_parquet/, features/, macro/
  outputs/ models/, metrics/, plots/, shap/
```